



گزارش فاز ۸: تحول و نگهداری نرم افزار (مدیریت درخواست تغییر)

پروژه: سامانه مدیریت و پایش داروهای استراتژیک

معماری پایه: ۳ لایه (UI، منطق تجاری، دیتابیس مرکزی) + ماژول هشدار مستقل (Alert Module)

سناریوی درخواست تغییر (Change Request) در هفته نهم

نکته: (از دیدگاه خودمان به عنوان کارفرما تصور کردیم)

«با توجه به اهمیت زنجیره سرما (Cold Chain) در نگهداری برخی داروهای حساس استراتژیک (مانند واکسن ها و انسولین)، سامانه باید بتواند داده های سنسورهای دمای انبارها را به صورت آنی دریافت کند. در صورت تغییر دما خارج از محدوده مجاز، سیستم باید فوراً از طریق ماژول هشدار به مدیران انبار پیامک فرستاده و وضعیت را در داشبورد ثبت کند.»

تحلیل تأثیر تغییر (Impact Analysis)

اعمال این نیازمندی جدید، بخش های مختلف معماری ۳ لایه و ماژول هشدار ما را به شرح زیر تحت تأثیر قرار می دهد:

الف) تأثیر بر نیازمندی ها (Requirements)

1. افزودن نیازمندی های وظیفه مندی جدید:

1. ثبت بازه دمای مجاز برای هر دارو در سیستم.
2. دریافت و پردازش داده های ارسالی از سنسورها (شبیه سازی با API).
3. ثبت تاریخچه دماهای غیرمجاز در پایگاه داده.

2. تغییر در نیازمندی‌های غیروظیفه‌مندی:

1. افزایش نرخ درخواست‌ها به سرور (به دلیل ارسال متناوب داده‌های دما).
2. امنیت داده‌های سنسورها جهت جلوگیری از ثبت داده‌های مخرب یا فیک.

ب) تأثیر بر طراحی و معماری (Architecture & Design)

1. **لایه نمایش (Presentation Tier):** نیاز به اضافه کردن بخش تعریف بازه دمایی در فرم ثبت دارو، و طراحی یک ویجت نمودار دمایی لحظه‌ای در داشبورد مدیر انبار.
2. **لایه منطق تجاری (Business Tier):** اضافه شدن یک سرویس جدید به نام TemperatureMonitorService برای ارزیابی مداوم دما و مقایسه آن با بازه استاندارد دارو.
3. **لایه داده (Data Tier):** افزودن ستون‌های min_temp و max_temp به جدول Drugs.
- ✓ ایجاد جدول جدید TemperatureLogs برای ثبت تاریخچه دما (دارای فیلدهای: شناسه دارو، شناسه انبار، دمای ثبت شده، و زمان ثبت).
4. **ماژول مستقل هشدار (Alert Module):** این ماژول که قبلاً برای هشدارهای کاهش موجودی استفاده می‌شد، حالا باید به یک ساب‌سکرایپر جدید برای رویدادهای دمایی مجهز شود تا بتواند قالب پیامک هشدار دما را تولید و ارسال کند.

ج) تأثیر بر کد موجود (Code Base)

1. تغییر در مدل‌های داده در بک‌اند (کدهای Flask/Django یا هر فریم‌ورک مورد استفاده).
 2. نوشتن مایگريشن دیتابیس برای اعمال تغییرات بدون پاک شدن داده‌های قبلی داروهای ثبت شده.
 3. پیاده‌سازی یک Endpoint جدید در API برای دریافت داده‌های دما از کلاینت/سنسور شبیه‌سازی شده.
-

تخمین هزینه اعمال تغییر (Risk Estimation & Cost)

تخمین ما برای پیاده‌سازی این درخواست تغییر به شرح زیر است:

الف) تخمین تلاش و زمان (Time & Effort)

کل زمان مورد نیاز برای پیاده‌سازی و تست این کار حدود 18 تا 22 ساعت کاری برآورد می‌شود که بین اعضای تیم به شکل زیر تقسیم می‌شود:

1. تغییرات دیتابیس و نوشتن مایگريشن: 2 ساعت
2. توسعه منطق پردازش دما در بک‌اند و اتصال به مازول هشدار: 6 ساعت
3. طراحی رابط کاربری و نمودار دمایی در فرانت‌اند: 6 ساعت
4. نوشتن تست‌های واحد (Unit Tests) و تست یکپارچگی: 4 ساعت
5. مستندسازی و دیپلوی مجدد: 2 ساعت

ب) تحلیل ریسک (Risk Analysis)

ریسک ۱ (بالا بودن حجم داده‌های دما): اگر سنسورها در ثانیه چندین بار داده بفرستند، دیتابیس مرکزی ما زیر بار می‌رود.

راهکار کاهش ریسک: سنسورها فقط در صورت تغییر دما یا هر ۵ دقیقه یک‌بار داده ارسال کنند. مگر اینکه دما به محدوده بحرانی نزدیک شود.

ریسک ۲ (تداخل در کارکرد مازول هشدار): تغییر در ساختار مازول هشدار برای پشتیبانی از پیامک دما ممکن است بخش هشدارهای قبلی (موجودی دارو) را خراب کند.

راهکار کاهش ریسک: استفاده از تست‌های رگرسیون (Regression Testing) روی مازول هشدار قبل از ادغام نهایی کد.

طرح بازطراحی (Redesign Plan)

برای تطبیق سیستم با این نیازمندی، تغییرات زیر در طراحی اعمال می‌شود:

الف) نحوه ارتباط اجزا (Sequence)

1. کلاینت/سنسور دما داده را به Endpoint `api/v1/temperature/log/` می‌فرستد.
2. لایه کنترلر بک‌اند داده را دریافت کرده و به `TemperatureMonitorService` می‌دهد.
3. این سرویس بازه مجاز دارو را از دیتابیس می‌خواند.
4. در صورت نامعتبر بودن دما، یک Event به نام `TemperatureOutOfRange` به صورت ناهمگام (Asynchronous) به سمت ماثول هشدار مستقل (Alert Module) شلیک می‌شود.
5. ماثول هشدار مستقل به صورت موازی پیامک هشدار را برای سرپرست انبار ارسال کرده و لاگ خطا را در دیتابیس ثبت می‌کند.

ب) تغییر در نمودار کلاس‌ها (Class Diagram Updates)

کلاس `Drug` فیلدهای مربوط به آستانه دما را دریافت می‌کند و کلاس جدید `TemperatureLog` با ارتباط ۱ به چند به کلاس `Drug` متصل می‌شود. همچنین متد `checkTemperature()` به کلاس‌های پردازشی سیستم اضافه خواهد شد.

بازاندیشی در طراحی اولیه (Design Retrospective)

اگر ما این نیازمندی را از ابتدا می‌دانستیم، چطور سیستم را طراحی می‌کردیم؟

1. **معماری رویدادمحورتر:** در طراحی اولیه، ما ماژول هشدار را به عنوان یک ماژول مستقل دیدیم اما ارتباط آن با سیستم اصلی تا حدی جفت‌شده (Coupled) بود.

✓ اگر از روز اول می‌دانستیم قرار است داده‌های سنسوری و آنی پردازش شوند، حتماً از یک (Message Broker) مثل RabbitMQ یا حداقل معماری Pub/Sub مبتنی بر Redis استفاده می‌کردیم تا ماژول هشدار کاملاً بدون وابستگی به وب‌سرور کار کند.

2. **دیتابیس مجزا برای لاگ‌ها:** ذخیره کردن لاگ‌های دمایی در دیتابیس رابطه‌ای مرکزی در طولانی‌مدت کارایی را پایین می‌آورد.

✓ بهتر بود از ابتدا داده‌های سری زمانی (Time-Series Data) را در یک دیتابیس NoSQL مثل MongoDB یا InfluxDB ذخیره می‌کردیم و فقط موجودی اصلی و مشخصات داروها را در دیتابیس رابطه‌ای نگه می‌داشتیم.

نتیجه‌گیری

طراحی سه لایه فعلی ما انعطاف‌پذیری لازم برای این تغییر را دارد، اما برای مقیاس‌های بزرگ، نیاز به حرکت به سمت الگوهای توزیع‌شده‌تر حس می‌شود.